

uniSaps - Documentation technique

Auteur : Marius Durand - Master 1 CNAM, parcours TRIED

Version application : 1.0.1+4

Version API : 1.0.0

Projet Firebase : [unisaps-3ad84](#)

1. Contexte et besoin

1.1 Contexte

Ce projet m'a particulièrement motivé car il rassemble deux domaines que j'apprécie énormément : le développement informatique et la mode.

Je suis personnellement passionné par les vêtements, les tenues et l'univers du style en général. J'aime aussi partager cette passion, regarder des inspirations, créer des outfits et réfléchir à l'image que peut transmettre une tenue. Le fait de pouvoir combiner cette passion avec mes études m'a donné beaucoup plus de motivation qu'un projet "classique" uniquement scolaire.

C'est donc dans cette logique que j'ai créé **uniSaps**, une application de dressing interactif mêlant gestion de garde-robe, création de tenues, recommandations, réseau social et même une réflexion autour de la monétisation et des comptes créateurs. Mon objectif était de construire un vrai projet cohérent et évolutif, et pas seulement une démonstration technique réalisée pour les cours.

J'ai essayé de penser le projet comme une application pouvant réellement être utilisée sur le long terme, avec une architecture propre, des fonctionnalités complètes et un modèle qui pourrait être déployé en production.

Avant le développement, j'ai aussi pris le temps de réfléchir aux problèmes que beaucoup de personnes rencontrent autour de la mode et de l'organisation de leur dressing :

- "Comment je m'habille ce matin ?"
- manque d'inspiration ou difficulté à créer des tenues,
- vêtements oubliés dans le dressing,
- absence de vue d'ensemble sur ses habits,
- envie de partager son style avec d'autres personnes.

Toutes les fonctionnalités de uniSaps ont ensuite été pensées pour répondre à ces problématiques : gestion numérique des vêtements, suggestions de tenues, système de swipe, historique des outfits, réseau social Inspiration, statistiques d'utilisation et fonctionnalités IA.

1.2 Besoin utilisateur

Je pars du constat que les utilisateurs veulent :

1. **Centraliser** leur garde-robe numériquement (photos, métadonnées).
2. **Décider plus vite** quoi porter (outfit du jour, météo, historique).
3. **Motiver** l'usage régulier (streak, stats).
4. **Partager** leurs looks dans un cercle social contrôlé (amis) ou ouvert (explorer).

Le projet uniSaps a été pensé autour de plusieurs profils d'utilisateurs ayant des usages différents de la mode et des réseaux sociaux vestimentaires.

Persona 1 — Passionné de mode

Le premier profil correspond à un utilisateur passionné par la mode, qui possède déjà un style affirmé et souhaite partager ses tenues avec une communauté.

Pour ce type d'utilisateur, l'application sert à la fois de dressing numérique et de réseau social spécialisé.

Les fonctionnalités importantes pour ce persona sont :

- l'ajout rapide de vêtements avec photos et métadonnées,
- la création de tenues complètes,
- le partage de posts dans l'onglet Inspiration,
- les statistiques d'utilisation et le streak,
- la personnalisation du profil et la gestion des amis.

L'objectif est de permettre à cet utilisateur de valoriser son style, retrouver facilement ses pièces et publier régulièrement ses outfits dans un environnement centré sur la mode.

Persona 2 — Débutant en mode

Le deuxième profil représente un utilisateur moins expérimenté dans le domaine de la mode, qui cherche principalement de l'inspiration et de l'aide pour composer des tenues.

Ce persona utilise davantage :

- le feed Explorer pour découvrir des styles,
- les suggestions de tenues,
- le système de swipe,
- les recommandations liées à la météo et aux saisons,
- les fonctionnalités IA pour analyser automatiquement les vêtements.

L'objectif est de réduire le temps nécessaire pour choisir une tenue et de rendre l'expérience plus accessible même pour des utilisateurs n'ayant pas de connaissances particulières en mode.

Persona 3 — Marque de vêtements indépendante

Le troisième profil correspond à une petite marque ou un créateur indépendant souhaitant présenter ses collections à une audience ciblée.

Pour ce besoin, uniSaps intègre un système de comptes Créateur permettant :

- la publication de posts sponsorisés,
- la mise en avant de collections dans le feed Explorer,
- la gestion d'un profil de marque,
- le lien entre compte personnel et compte créateur.

Cette fonctionnalité apporte une dimension plus professionnelle au projet et permet d'imaginer un modèle économique basé sur la visibilité des marques émergentes au sein de la plateforme.

1.3 Analyse de l'existant

Avant de concevoir uniSaps, j'ai analysé plusieurs types d'applications déjà présentes sur le marché afin d'identifier leurs points forts mais aussi leurs limites.

Applications centrées sur l'inspiration (Pinterest, Instagram)

Des plateformes comme Pinterest ou Instagram permettent de découvrir énormément d'inspirations vestimentaires et de suivre des créateurs de contenu mode.

Cependant, ces applications restent principalement centrées sur le partage de contenu :

- aucune vraie gestion de garde-robe personnelle,
- pas d'organisation des vêtements,
- difficulté à retrouver précisément les pièces utilisées dans une tenue,
- absence de recommandations adaptées au dressing réel de l'utilisateur.

L'utilisateur peut trouver de l'inspiration, mais il ne dispose pas d'outil concret pour gérer ses propres vêtements ou construire facilement ses outfits.

Applications de gestion de dressing

J'ai également regardé des applications spécialisées dans la gestion de garde-robe numérique.

Ces solutions proposent généralement :

- l'ajout de vêtements,
- des catégories simples,
- parfois la création de tenues.

Mais elles restent souvent limitées à un usage "catalogue" :

- peu ou pas de dimension sociale,
- expérience utilisateur parfois vieillissante,
- absence de recommandations intelligentes,
- peu d'interactions ou de motivation à revenir régulièrement sur l'application.

La plupart ne proposent pas non plus d'intégration IA ou de logique avancée autour du style, de la météo ou de l'historique des vêtements portés.

Positionnement de uniSaps

Avec uniSaps, l'objectif était de proposer une expérience plus complète et plus moderne en regroupant plusieurs usages dans une seule application.

Le projet combine :

- une gestion complète du dressing,
- un système de création de tenues,
- un réseau social mode,
- des suggestions intelligentes basées sur l'IA et des règles de scoring,

- des statistiques et systèmes d'engagement (streak, historique, usage des vêtements).

L'idée principale est donc de proposer une application "tout-en-un" qui ne sert pas uniquement à regarder des inspirations, mais aussi à organiser son dressing, créer ses tenues et partager son style avec une communauté.

2. Fonctionnalités (plus détaillé dans [./README.md](#))

2.1 Authentification et onboarding

- **Firestore Auth** : inscription et connexion email/mot de passe.
- **Onboarding** : pseudo, photo de profil après première connexion.
- **Tutoriels** : bulles contextuelles par onglet (`tutorial_seen.*` dans Firestore), affichage unique, taille limitée (~35 % hauteur écran), boutons Suivant / Passer.
- **Responsive** : écrans login/signup adaptés aux largeurs ≥ 320 px (textes courts, retours à la ligne).

2.2 Dressing (bibliothèque de vêtements)

Fonction	Détail technique
CRUD vêtements	Sous-collection <code>users/{uid}/garments</code>
Catégories	<code>top, bottom, shoes, outerwear, headwear, accessory</code>
Photos multiples	Champ <code>image_urls[]</code> ; upload via API backend (resize JPEG) ou Storage direct
Marques	Liste embarquée <code>assets/data/brand.json</code> + saisie libre
Couleurs	Palette normalisée côté client (<code>color_service.dart</code>)
Analyse IA	<code>POST /api/v1/ai/analyze-garment</code> - réservé premium

Flux ajout vêtement : photo → (option IA) → préremplissage formulaire → validation → Firestore + Storage.

2.3 Outfits (tenues et outfit du jour)

Fonction	Détail
Composition	6 zones : <code>headwear, top, outerwear, bottom, shoes, accessory</code>
Modes de sélection	Bibliothèque (scroll horizontal) et Swipe (<code>flutter_card_swiper</code>)
Outfit du jour	Champs user : <code>daily_outfit_id, daily_outfit_date, daily_photo_url</code>
Streak	<code>current_streak, best_streak</code> ; logique client au changement de jour
Historique de port	<code>times_worn, last_worn, wear_history[]</code> sur chaque outfit
Météo	Open-Meteo + géolocalisation ; tags <code>weather_tags</code> et <code>seasons</code> pour tri
Suggestions IA	<code>POST /api/v1/ai/suggest</code> - 3 propositions, prompt + contexte météo/saison

Niveau 1 (aucun outfit du jour) : bannière d'état + tabs Bibliothèque / Swipe / IA (sheet premium).

Niveau 2 (outfit choisi) : affichage principal, photo optionnelle, streak visible.

2.3bis Navigation principale

- **4 onglets** dans `HomeScreen` : Dressing (0) · Outfits (1) · Inspiration (2) · Profil (3).
- **Création de tenue** : FAB « Ajouter un fit » sur Outfits (mode Bibliothèque) → `creation_screen.dart` (pas d'onglet « Créer »).
- Verrouillage : Outfits et Inspiration tant qu'il n'y a pas au moins un vêtement / un outfit.

2.4 Inspiration (réseau social)

- Collection racine `posts/{postId}`.
- Feeds **Explorer** (tous les posts) et **Amis** (filtrage `friends[]`).
- Like atomique (`liked_by`, `likes`) - règles Firestore `onlyLikeChanged()`.
- **1 post par jour** : vérification `getUserTodayPost` côté client.
- Références vêtements sur le post (`garment_refs`).
- Recherche utilisateurs, demandes d'amis (`friend_requests`).

2.5 Profil

- Sous-sections : statistiques agrégées, galerie photos du jour, informations compte.
- Compte **privé** (`is_private`) : contrôle de visibilité future.
- **UniSaps+** : `account_tier` = `premium` | `free` ; voir section monétisation ci-dessous.

2.6 Monétisation (UniSaps+ et Créateurs)

UniSaps+ - offre utilisateur (en place)

J'ai défini et intégré deux formules pour débloquer l'IA :

Formule	Prix	Fonctionnalités
Abonnement mensuel	1 € / mois	<code>POST /ai/analyze-garment</code> , <code>POST /ai/suggest</code> , badge premium
Achat à vie	5 € (unique)	Idem, sans renouvellement

Implémentation actuelle : champ Firestore `account_tier`, écrans d'upgrade (`premium_upgrade_dialog.dart`), section UniSaps+ dans le profil. L'activation en développement peut passer par un code ; le branchement **Google Play Billing** est prévu pour encaisser les tarifs en production.

Justification : les appels Gemini restent peu coûteux unitairement ; à 1 €/mois je finance l'API et je garde un positionnement accessible. L'offre à 5 € récompense les premiers utilisateurs et améliore ma trésorerie au lancement.

Comptes Créateur - offre marques (MVP implémenté)

- **Cible** : marques indépendantes ; parcours `/creator` → checkout stub → onboarding → shell 3 onglets.
- **Modèle** : abonnement mensuel affiché (29 €/mois) ; activation dev via code `CREATOR2026` ou essai stub (Google Play Billing en phase 2).

- **Données** : `account_type: creator`, collections, posts `post_kind: sponsored`, `is_active`, `preview_image_url`, `linked_user_uid` vers le compte perso du responsable.
- **Feed Explorer** : mixage client 1 sponsorisé / 7 organiques ; badge **Sponsorisé** sur `PostCard` ; pas de mélange dans le feed Amis.
- **Backend** : `POST /api/v1/creator/activate-subscription`, `GET /api/v1/creator/subscription-status`.

2.7 Widgets Android (bonus)

- `home_widget` : synchronisation outfit du jour, inspiration, choix rapide.
- Code natif Kotlin dans `frontend/android/.../widgets/`.

3. Choix techniques

3.1 Stack retenue

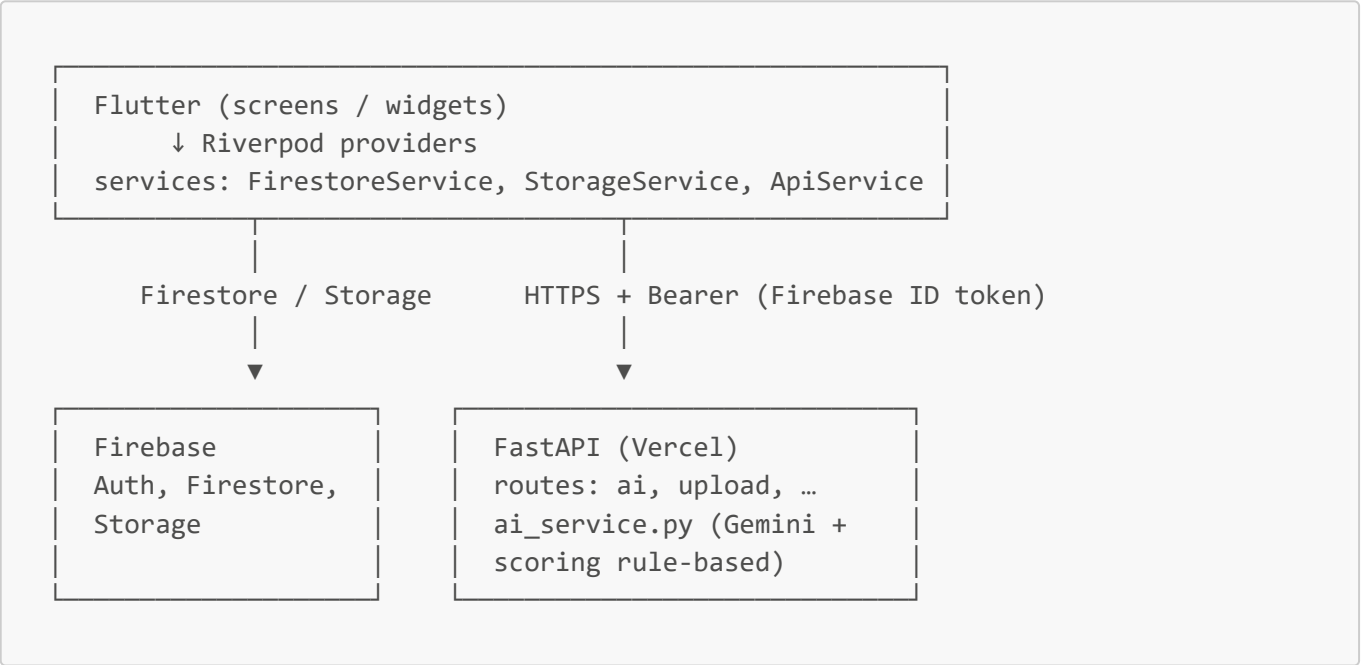
Couche	Technologie	Justification
Mobile	Flutter 3.24+	UI native performante, un seul codebase, écosystème widgets (swipe, cache images)
State	Riverpod 2	Réactivité, testabilité (providers injectables)
Navigation	go_router	Routes auth + deep links
BDD temps réel	Cloud Firestore	Sync offline, règles fines, sous-collections par user
Fichiers	Firebase Storage	URLs signées, intégration Auth
Auth	Firebase Authentication	Standard marché, token JWT pour API
API métier	FastAPI (Python 3.11)	PO Java/Python, typage Pydantic, déploiement serverless
Hébergement API	Vercel	HTTPS gratuit, cold start acceptable pour IA ponctuelle
Météo	Open-Meteo	Gratuit, sans clé, suffisant pour tags météo
Vision	Google Gemini 2.5 Flash	Multimodal, tier gratuit génèreux, JSON structuré
Suggestions	Moteur rule-based Python	Latence < 1 s, coût nul, explicable devant jury

3.2 Alternatives écartées

Option	Raison de l'écart
--------	-------------------

Option	Raison de l'écart
Flet (Python UI) - V1	Peu de widgets mobiles matures, perf et responsive limités → abandon
React Native	Moins aligné avec exigence Python significatif côté backend
Supabase seul	Moins intégré écosystème mobile Firebase déjà utilisé en cours
LLM pour chaque suggestion d'outfit	Coût et latence (3–5 s difficiles à tenir à grande échelle)
GPT-4 Vision	Coût par image supérieur à Gemini Flash pour un projet étudiant
Parse / Backendless	Vendor lock-in, moins de contrôle sur les règles Firestore

3.3 Architecture logicielle



Pattern : pas de couche « Controller » MVC classique ; les **providers Riverpod** portent la logique métier (équivalent contrôleurs).

Organisation frontend :

- `lib/models/` - sérialisation Firestore
- `lib/services/` - accès données et HTTP
- `lib/providers/` - état et orchestration
- `lib/screens/` - vues
- `lib/widgets/` - composants réutilisables
- `lib/core/` - thème, routes, constantes

Organisation backend :

- `app/api/routes/` - endpoints REST
- `app/models/` - schémas Pydantic
- `app/services/` - `ai_service.py`, `storage_service.py`

- `app/core/` - config Firebase, sécurité JWT

3.4 POO et qualité de code

- **Modèles** : classes Dart immutables avec `fromFirestore` / `toMap` ; miroirs Pydantic côté API.
- **Services** : responsabilité unique (auth, firestore, api, météo).
- **Providers** : composition et invalidation ciblée (`ref.invalidate`).
- **Python** : fonctions pures de scoring (`_score`, `suggest_outfit`) testables sans UI.
- **Type hints** Python et typage Dart strict sur les API publiques.

4. Section IA

4.1 Usages de l'IA dans le projet

Usage	Outil / modèle	Rôle
Génération de code	Cursor, agents Composer	Scaffolding Flutter/Python, refactor
Analyse photo vêtement	Gemini 2.5 Flash	Vision → JSON structuré
Suggestions d'outfits	Algorithme déterministe (pas LLM)	Score couleurs, style, saison, météo, historique
Documentation	LLM	README, rapport, cette doc
Suppression de fond	rembg (optionnel, local)	Désactivé en prod Vercel

4.2 Analyse d'image (Gemini)

Fichier : `backend/app/services/ai_service.py` - `analyze_garment_image()`.

- Entrée : bytes image JPEG/PNG.
- Prompt : JSON strict (`is_garment`, `name`, `brand`, `colors`, `category`, `style_tags`, `formality`, `season`, `pattern`, `material`).
- Température **0.2** pour limiter l'hallucination.
- Rejet si `is_garment == false` (photo non vestimentaire).
- Côté client (`add_garment_sheet.dart`) : retry si réponse vide, réservé premium.

Extrait de prompt (réel) :

```
Tu es un assistant de mode qui analyse UNE SEULE pièce vestimentaire sur une photo.
Réponds STRICTEMENT au format JSON suivant, sans texte autour :
{ "is_garment": true, "name": "...", "brand": "...", "colors": [...], ... }
- "colors" : noms français parmi une palette fermée (Noir, Bleu marine, Beige, ...)
```

4.3 Suggestions d'outfits (rule-based)

Fichier : backend/app/services/ai_service.py - suggest_outfit(), suggest_multiple().

- 1. **Mapping prompt :** PROMPT_MAP traduit « classe », « bureau », « streetwear »... vers un style canonique (Classe, Professionnel, Streetwear...).
- 2. **Familles de couleurs :** COLOR_FAMILIES + STYLE_WEIGHTS pour noter la cohérence chromatique.
- 3. **Attributs IA garment :** bonus si style_tags, formality, season, material alignés avec le style et la météo.
- 4. **Anti-répétition :** pénalité -2 si pièce portée dans les 7 derniers jours (last_worn sur outfits existants).
- 5. **Météo :** si tags cold, rain, snow → force une outerwear si disponible.
- 6. **Variété :** suggest_multiple() déduplique les combinaisons.

Appel client (outfits_screen.dart) :

```
final list = await api.suggestOutfits(
  style: style,
  count: 3,
  seasonKey: ctx.seasonKey,
  weatherTags: ctx.activeWeatherTags.toList(),
);
```

4.4 Analyse des coûts API IA

Service	Tarification indicative	Usage uniSaps	Choix
Gemini 2.5 Flash	Tier gratuit (~1500 req/jour en dev)	1 appel / ajout vêtement premium	Retenu - meilleur rapport qualité/coût pour la vision
GPT-4o mini / Vision	~0,15–0,60 \$ / M tokens + image	Équivalent fonctionnel	Écarté - coût cumulé si tous les users analysent chaque photo
Claude Vision	Payant à l'usage	Idem	Écarté - même raison
LLM par suggestion outfit	1–3 appels / matin / user	Centaines de tokens + latence	Écarté - moteur rule-based gratuit et < 1 s

Estimation ordre de grandeur (100 users actifs, 5 vêtements analysés/mois) :

≈ 500 appels Gemini/mois → reste dans le gratuit ou quelques euros. Un LLM par suggestion multiplierait par 30× les appels.

J'ai donc du faire des choix par rapport à ces coups au seins de l'app pour correspondre et à mon budget de casi 0 euro et la garantie d'avoir des fonctionnalités optimal pour l'utilisateur.

4.5 Workflow agents Cursor (développement)

Développement assisté par **trois agents spécialisés** invocables dans le chat Cursor via @ :

Mention @	Fichier prompt	Mission	Livrables principaux
unisaps-backend	.cursor/agents/unisaps-backend.md	FastAPI métier (hors /ai/*), JWT, upload, creator, firestore.rules, Vercel	backend/app/ (sauf ai.py, ai_service.py), firestore.rules
unisaps-ia	.cursor/agents/unisaps-ia.md	Gemini vision, scoring rule-based, routes /ai/*, garde UniSaps+, intégration client IA	ai_service.py, ai.py, api_service.dart, add_garment_sheet.dart, outfits_screen.dart
unisaps-frontend	.cursor/agents/unisaps-frontend.md	UI/UX mobile, thème, écrans, widgets, Riverpod, Firestore client, 320 px	frontend/lib/, core/theme, widgets

Index, matrice de périmètres et exemples : [.cursor/AGENTS.md](../.cursor/AGENTS.md).

Séparation IA / backend : @unisaps-backend ne modifie pas ai_service.py ni api/routes/ai.py ; @unisaps-ia porte toute la chaîne vision + suggestions + premium. Le client Flutter utilise Firestore pour le CRUD ; l'API sert surtout à l'IA et à l'upload.

Comment choisir son agent

- Endpoint REST, règles Firestore, Storage, déploiement Vercel → **@unisaps-backend**
- Analyse photo, _score(), suggestions, Gemini, dialog premium → **@unisaps-ia**
- Écran, widget, navigation, design, feed, streak, swipe → **@unisaps-frontend**
- Tâche mixte : commencer par l'agent dominant, puis enchaîner un second @ si nécessaire

Exemples de prompts par agent

Backend — @unisaps-backend Durcis POST /creator/activate-subscription : vérifie le token Firebase, écrit account_type: creator et renvoie un JSON aligné avec le modèle user côté Flutter.

IA — @unisaps-ia Dans ai_service.py, ajoute un bonus _score() quand material du vêtement correspond à la saison active ; garde 3 suggestions via suggest_multiple et la garde premium sur POST /ai/suggest.

Frontend — @unisaps-frontend Sur outfits_screen.dart, améliore l'onglet Swipe : carte tenue pleine largeur, pas d'overflow à 320 px, bouton « Choisir pour aujourd'hui » visible après le swipe.

Exemple de prompt « chef de projet » (cahier des charges initial) :

```
# Application de Gestion de Garde-Robe (Outfit Manager)
## Objectif
Développer une application mobile avec Firebase : photographier les vêtements,
créer des tenues, outfit du jour, streak, réseau social Inspiration.
## Navigation : Dressing | Outfits | Inspiration | Profil
```

```
La création de tenue passe par le **FAB « Ajouter un fit »** sur l'onglet Outfits  
(écran creation_screen.dart`), pas par un onglet dédié.  
[... contraintes responsive, IA, streak 00h00, structure /models /services ...]
```

Découper ensuite par agent : frontend pour la navigation et les écrans, IA pour Gemini et le scoring, backend pour les routes métier et `firestore.rules`.

4.6 Construction de plan IA (fonction "Plan" de Cursor)

En complément des agents spécialisés, j'ai utilisé la fonctionnalité **"Plan" de Cursor** avant certaines implémentations importantes. Cette fonctionnalité permet de demander à l'IA non pas directement du code, mais une **structuration du besoin**, une analyse des contraintes et un découpage du travail en étapes cohérentes.

L'objectif était d'éviter un développement "au fil de l'eau" où l'IA génère rapidement du code sans vision globale de l'architecture.

Un exemple de ce que le mode plan pourrait renvoyer est présent dans le dossier `.cursor/plan/...`

Structuration du besoin

Avant d'implémenter une fonctionnalité complexe (exemple : système d'outfit du jour, feed social ou comptes créateur), je formulais le besoin dans Cursor en décrivant :

- les fonctionnalités attendues,
- les contraintes techniques,
- les données Firestore concernées,
- les impacts UI / backend,
- les règles métier.

La fonctionnalité "Plan" produisait alors :

- un découpage en sous-tâches,
- une proposition d'architecture,
- les fichiers potentiellement impactés,
- les risques techniques,
- l'ordre recommandé des développements.

Cela m'a permis d'avoir une approche plus proche d'une démarche d'ingénierie logicielle classique plutôt qu'un simple usage de génération automatique de code.

Discussion sur le besoin

J'ai également utilisé l'IA comme outil de discussion technique.

Avant certaines implémentations, je confrontais plusieurs approches possibles :

- logique côté client ou backend,
- Firestore direct ou API intermédiaire,
- IA générative ou moteur déterministe,
- données calculées dynamiquement ou stockées.

Le mode "Plan" servait alors de support de réflexion.

L'intérêt principal n'était pas uniquement la réponse finale, mais le raisonnement proposé par l'IA : avantages, limites, impacts sur les coûts, la maintenabilité ou les performances.

Par exemple, pour les suggestions d'outfits, cette phase de réflexion m'a conduit à abandonner une approche 100 % LLM au profit d'un moteur rule-based beaucoup plus économique et beaucoup plus rapide en interface surtout.

Questions de l'agent

Un point particulièrement utile a été la capacité de Cursor à poser des questions avant génération.

Lorsque le besoin était ambigu ou incomplet, l'agent demandait par exemple :

- quelles collections Firestore utiliser,
- si la logique devait être temps réel,
- quelles contraintes responsive appliquer

Cela m'a obligé à préciser plusieurs décisions techniques en amont, exactement comme dans un échange avec un chef de projet ou un lead développeur.

Cette étape a réduit les générations incohérentes et amélioré la qualité globale du code produit.

4.7 Gestion multi-agents et développement simultané avec Cursor

En plus des agents spécialisés, j'ai utilisé les fonctionnalités avancées de gestion d'agents de Cursor pour travailler sur plusieurs sujets en parallèle.

L'objectif était de se rapprocher d'une organisation "multi-équipe" où plusieurs agents IA interviennent chacun sur un périmètre précis, parfois simultanément sur différentes branches Git et avec des expertises différentes selon le besoin et l'agent choisi.

Agents simultanés

Cursor permet de lancer plusieurs conversations d'agents spécialisées en parallèle.

Dans mon workflow, cela signifiait par exemple :

- un agent travaillant sur l'UI Flutter,
- un autre sur les routes FastAPI,
- un troisième sur la logique IA et le scoring.

Chaque agent conservait son contexte technique et ses contraintes.

Cela évitait de mélanger plusieurs responsabilités dans une seule conversation IA, ce qui améliore fortement la cohérence des réponses.

Cette séparation se rapproche d'une organisation réelle en équipe de développement avec plusieurs profils spécialisés.

Travail multi-branche

J'ai également utilisé des branches Git séparées selon les sujets :

- `feature/frontend-*`
- `feature/backend-*`
- `feature/ai-*`

Les agents Cursor intervenaient alors sur des branches différentes selon leur rôle.

Cette méthode apportait plusieurs avantages :

- isolation des modifications,
- réduction des conflits,
- possibilité de tester une fonctionnalité sans casser le reste du projet,
- comparaison plus simple entre plusieurs implémentations proposées par l'IA.

Cela m'a aussi permis de conserver un historique Git plus propre et plus professionnel.

Management des agents

Une partie importante du travail consistait à "manager" les agents IA.

Concrètement, je devais :

- définir précisément le périmètre de chaque agent,
- rédiger des consignes claires,
- vérifier les modifications proposées,
- arbitrer entre plusieurs solutions,
- corriger les incohérences entre agents.

L'IA ne remplace donc pas la prise de décision technique : elle accélère l'exécution, mais nécessite un pilotage humain constant.

J'ai remarqué que la qualité des résultats dépendait énormément de la précision des prompts en général et du model choisi pour exécuter la tâche. En effet, n'ayant pas un compte cursor très haut gradé je n'ai pas des tokens infini. Il fallait donc optimiser l'utilisation des token et donc choisir des modèles moins puissant pour des tâche classique, plus puissant pour des grosses fonctionnalités etc... Chaque modèle à ses propres forces et faiblesses aussi. Pour la gestion des fonctionnalités IA j'utilise des appels API vers Google AI donc j'utilisais des modèles gemini car plus calé sur le sujet. Pour des tâches de code pur je favorisait claude qui est bien supérieur aux autres modèles sur ce domaine. Pour les tâches de rédaction ou encore d'exécution de commande terminal ChatGPT excel particulièrement dans le domaine (d'après mon expérience et les benchmark), finalement pour les tâches simple j'utilisais composer 2 modèle de cursor qui est à moindre coût et assez bon dans tous les domaines.

5. Données et fichiers nécessaires

5.1 Schéma Firestore (résumé)

Voir [README.md](#) racine. Collections principales :

- `users/{uid} + garments/, outfits/`

- `posts/{postId}`
- `friend_requests/{requestId}`

5.2 Firebase Storage

```
garments/{user_id}/{filename}
outfits/{user_id}/{filename}
profiles/{user_id}/{filename}
posts/{user_id}/{filename}
```

5.3 Fichiers de configuration

Fichier	Rôle
<code>frontend/android/app/google-services.json</code>	Config Android Firebase
<code>firestore.rules</code>	Sécurité données
<code>firebase.json</code>	Déploiement règles
<code>backend/serviceAccountKey.json</code>	Admin SDK (gitignored)
<code>backend/.env</code>	<code>GEMINI_API_KEY</code> , bucket Storage
<code>backend/vercel.json</code>	Routing serverless

6. Installation, déploiement et tests

6.1 Installation

Détaillée dans `presentation/README.md` et `backend/LANCE_BACKEND.md`.

6.2 Déploiement

- **API** : push sur branche connectée à Vercel → `https://unisaps.vercel.app`
- **Règles Firestore** : `firebase deploy --only firestore:rules`
- **APK** : `flutter build appbundle --release` → Play Console

6.3 Tests

Type	Emplacement	Commande
Modèles Dart	<code>frontend/test/models/</code>	<code>flutter test</code>
Providers	<code>frontend/test/providers/</code>	idem
Services	<code>frontend/test/services/</code>	idem
CI	<code>.github/workflows/flutter_tests.yml</code>	analyze + test sur push

Manques : tests widget/E2E, tests `pytest` backend.

6.4 Scénarios de test manuels (reproductibles)

1. Utilisateur sans vêtement → onglets Outfits/Inspiration verrouillés.
 2. Ajout vêtement sans réseau → message d'erreur clair.
 3. Photo non vêtement + IA → retrait slot + dialogue.
 4. Deuxième post le même jour → refus.
 5. Changement de date système → recalcul streak (à valider).
-
-

7. Limites et évolutions

- Finaliser **Google Play Billing** pour les offres 1 € et 5 € et pour le compte créateur.
 - Finaliser le compte créateur puis démarcher des marques.
 - iOS release, Cloud Functions reset minuit, purge compte, tests E2E.
 - Affiner la partie de création de outfits via IA pour offrir vraiment un plus à l'app.
-